

Lab 04

Flow Control Instructions

Lab Objectives

By the end of this lab, students will be able to:

1. Understand how flow control instructions influence program execution.
2. Use comparison and jump instructions for decision-making in assembly language.
3. Implement conditional and unconditional branching.
4. Relate assembly-level branching to high-level constructs (if-else, loops).

Introduction

Flow control instructions allow the program to change its normal sequential execution. Instead of executing instructions one after another, the processor can **jump**, **repeat**, or **branch** depending on conditions. This is how **if statements**, **loops**, and **switches** in high-level languages are implemented at machine level.

The following instructions will be followed:

- **JMP** – Unconditional jump.
- **CMP** – Compare two operands and set flags.
- **Conditional Jumps:**
 - **JE / JZ** → Jump if equal / zero.
 - **JNE / JNZ** → Jump if not equal / not zero.
 - **JG / JL / JGE / JLE** → Jump if greater / less (signed).
 - **JA / JB / JAE / JBE** → Jump if above / below (unsigned).
- **LOOP** – Repeat a block until CX = 0.

Unconditional Jump: This program illustrates an unconditional jump. The JMP instruction transfers control to the label SKIP, causing the instruction in between to be bypassed. As a result, the value in AL remains 5.

```
MOV AL, 5
JMP SKIP
MOV AL, 9 ; this will be skipped
SKIP:
; AL = 5
```

Output

registers		0100:0006		0100:0000	
	H L				
AX	00 05	01000: B0 176		MOV AL, 05h	
BX	00 00	01001: 05 005		JMP 06h	
CX	00 00	01002: EB 235		MOV AL, 09h	
DX	00 00	01003: 02 002		HLT	
		01004: B0 176		NOP	
		01005: 09 009	TAB	NOP	
		01006: F4 244		NOP	
		01007: 00 144		NOP	

Another Example: Jump to desired instruction.

```
MOV AX, 5
JMP OTHER ; skip the unwanted instruction
MOV AX, 10 ; this line will never execute
OTHER:
ADD AX, 2 ; AX = 5 + 2 = 7
```

Output

registers		0100:000B		0100:000B	
	H L				
AX	00 07	01000: B8 184		MOV AX, 00005h	
BX	00 00	01001: 05 005		JMP 08h	
CX	00 00	01002: 00 000	NULL	MOV AX, 0000Ah	
DX	00 00	01003: EB 235		ADD AX, 00002h	
		01004: 03 003		NOP	
		01005: B8 184		NOP	
		01006: 0A 010	NEWL	NOP	
		01007: 00 000	HLT	NOP	

Block: In assembly, a block is simply a labeled section of code that groups related instructions.

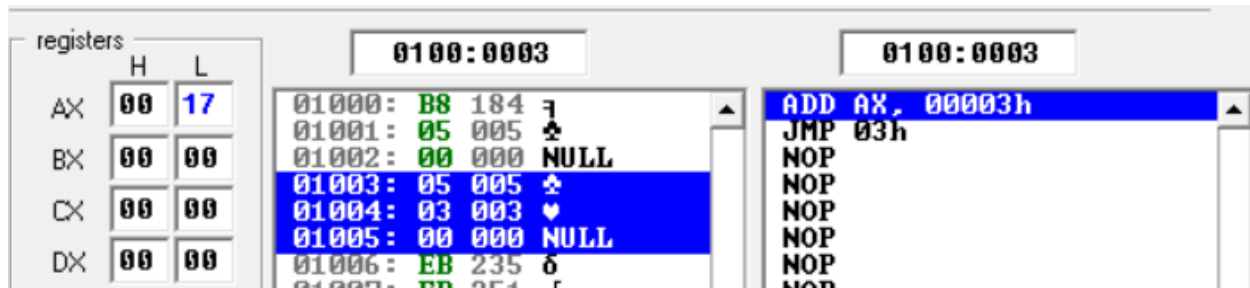
Example

```
START: MOV AX, 5 ; Start block
      ADD AX, 3
```

Practical Example

```
MOV AX, 5
START: ADD AX, 3 ; Start block
      JMP START
```

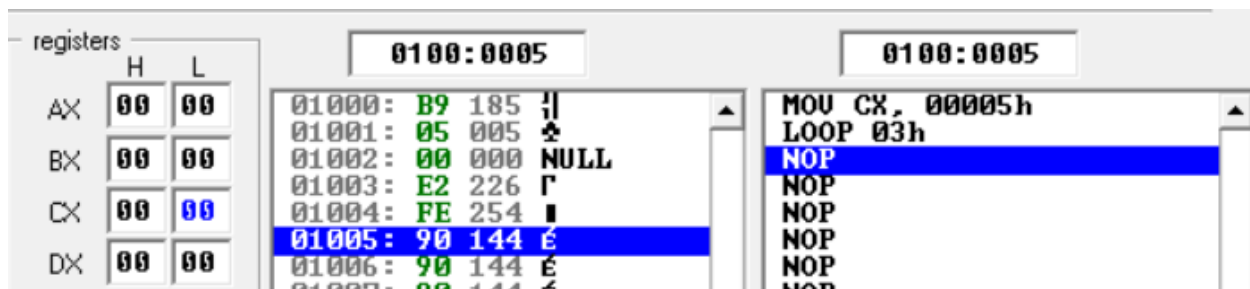
Output



Loop: In x86 assembly, the CX register is commonly used as a loop counter. The LOOP instruction is specifically designed to work with CX: each time LOOP is executed, the processor automatically decrements CX by 1 and continues looping until CX becomes zero.

```
MOV CX, 5 ; set loop counter to 5
TOP:
; (your code here, e.g., increment a register)
LOOP TOP ; CX = CX - 1, repeat until CX = 0
```

Output



Practical Example:

```
MOV AX, 0 ; Initialize AX = 0
MOV CX, 5 ; Set loop counter = 5
```

START:

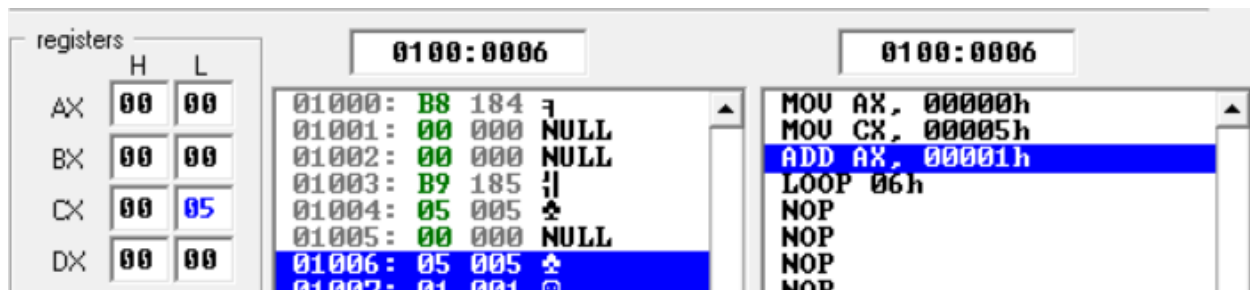
ADD AX, 1 ; Add 1 to AX

LOOP START ; Decrement CX, repeat until CX = 0

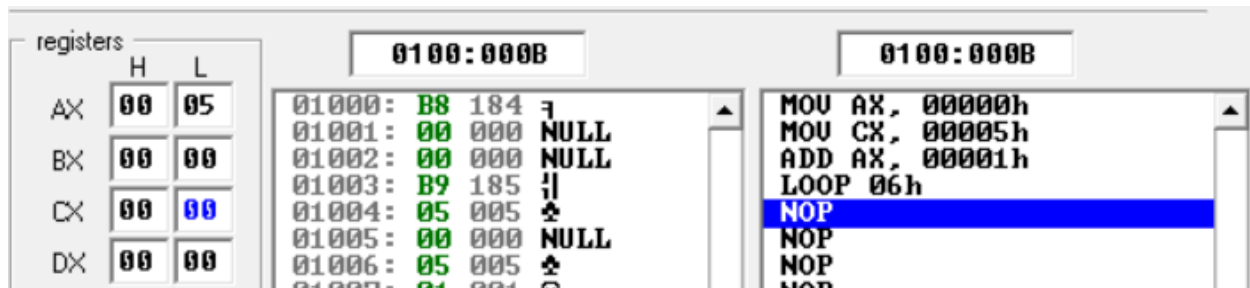
Explanation:

- AX starts at 0.
- CX is the counter (5 iterations).
- Each loop adds 1 to AX.
- After 5 repetitions, AX = 5 and CX = 0
- Program exits

CX and AX values at start.



CX and AX values at end.



Difference between JMP and LOOP:

JMP → repeats *forever* (or until you add your own condition).

LOOP → repeats *exactly CX times* (automatic counter).

Class work: Repeat above example but using SUB operation (Initialize AX = 10).

Conditions in assembly: Conditions in assembly are decisions based on CPU flags (Zero, Sign, Carry, etc.) set by instructions like CMP or arithmetic operations.

Conditional jumps (JE, JNE, JG, JL, etc.) test these flags to control program flow, enabling *if*, *else*, and *loop* structures.

Simple Example

```
MOV AX, 5      ; Store first number
MOV BX, 5      ; Store second number

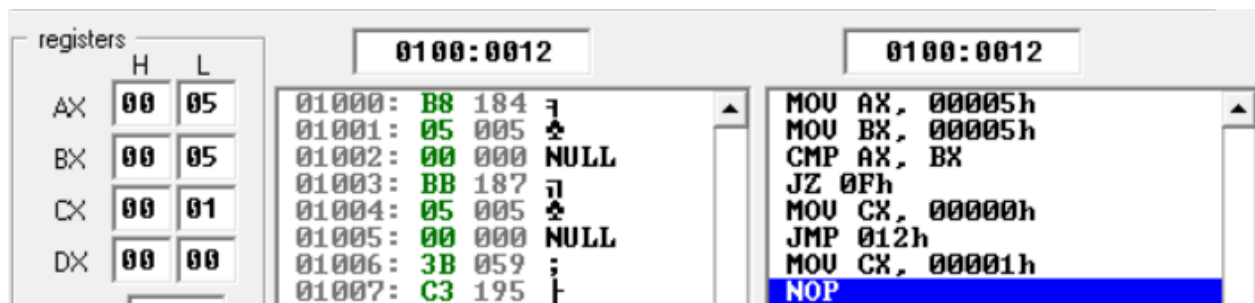
CMP AX, BX     ; Compare AX and BX
JE EQUAL       ; Jump if AX == BX

MOV CX, 0      ; Runs only if AX != BX
JMP END

EQUAL:
MOV CX, 1      ; Runs only if AX == BX

END:
; Program continues here
```

Output:



Combining Loop and Conditions:

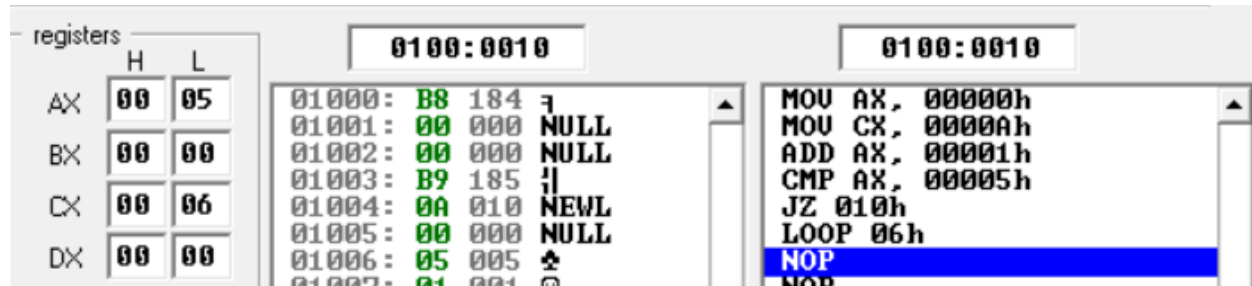
```
MOV AX, 0      ; Start value
MOV CX, 10     ; Loop counter = 10

START:
ADD AX, 1      ; Increase AX each iteration
CMP AX, 5      ; Compare AX with 5
JE FOUND       ; If AX = 5, jump to FOUND
LOOP START     ; Otherwise, continue loop
```

FOUND:

; AX = 5, program jumps here

Output



Explanation:

- AX increments each loop.
- CMP AX, 5 checks if AX has reached 5.
- When equal, JE breaks the loop early and jumps to FOUND.
- Otherwise, LOOP continues until CX = 0.

Task 1: Load AL = 2, then use JMP to skip over the instruction AL = 9. Verify AL still equals 2.

Task 2: Write a program that initializes AX = 0 and CX = 4, then uses LOOP to add 2 to AX each time.

Task 3: Write a program that compares AX and BX.

- If AX > BX, store 1 in DX.
- If AX < BX, store 2 in DX.
- If AX == BX, store 3 in DX.

Task 4: Write an assembly program that starts with AX = 10 and CX = 5.

- In each iteration, decrement AX by 1 (DEC AX).
- Use CMP to check if AX has reached 7.
- If equal, jump to a label STOP and end the loop early.